

# Split-Step Fourier Method for the Nonlinear Schrödinger Equation

PDE Course Notes

# The Nonlinear Schrödinger Equation (NLS)

We consider the 1D cubic NLS on a periodic domain  $x \in [-L, L]$ :

$$iu_t + u_{xx} + \kappa|u|^2u = 0$$

- ▶  $u(x, t)$  complex-valued
- ▶  $\kappa \in \mathbb{R}$  controls focusing/defocusing
- ▶ Periodic boundary conditions

We rewrite:

$$u_t = \underbrace{iu_{xx}}_{\text{linear}} + \underbrace{i\kappa|u|^2u}_{\text{nonlinear}}$$

# Splitting the PDE

We split into two parts:

$$u_t = L(u) + N(u)$$

where

$$L(u) = iu_{xx}, \quad N(u) = i\kappa|u|^2u$$

**Key idea:**

- ▶ Solve  $u_t = L(u)$  exactly (in Fourier space)
- ▶ Approximate  $u_t = N(u)$  explicitly (Euler)

# Strang Splitting

Second-order accurate splitting:

$$u^{n+1} = e^{\frac{\Delta t}{2}L} \circ \Phi_N^{\Delta t} \circ e^{\frac{\Delta t}{2}L}(u^n)$$

Steps:

1. Half linear step
2. Full nonlinear step
3. Half linear step

**Important:** symmetric  $\Rightarrow$  second order in time

# Linear Step: Fourier Representation

Take Fourier transform:

$$\hat{u}(k, t) = \mathcal{F}[u(x, t)]$$

Then:

$$\widehat{u_{xx}} = -(k^2)\hat{u}$$

So:

$$\hat{u}_t = -ik^2\hat{u}$$

# Symbol of the Linear Operator

The linear operator is:

$$L = i\partial_{xx}$$

Its Fourier symbol is:

$$\sigma_L(k) = -ik^2$$

Thus each Fourier mode satisfies:

$$\frac{d}{dt}\hat{u}(k, t) = \sigma_L(k)\hat{u}(k, t)$$

# Exact Linear Evolution

Each mode evolves independently:

$$\hat{u}(k, t + \Delta t) = e^{-ik^2 \Delta t} \hat{u}(k, t)$$

## Key point:

- ▶ Linear step is diagonal in Fourier space
- ▶ No approximation needed
- ▶ Just multiply by a phase factor

This is the main advantage of spectral methods.

# Nonlinear Step

We solve:

$$u_t = i\kappa|u|^2u$$

Using explicit Euler:

$$u^{n+1} = u^n + \Delta t i\kappa|u^n|^2u^n$$

## Notes:

- ▶ Computed in physical space
- ▶ Local (no derivatives)
- ▶ Cheap but conditionally stable

# Alternative ODE Solvers for the Nonlinear Step

We solve:

$$u_t = i\kappa|u|^2u$$

Explicit Euler is simple but it is unstable for large  $\Delta t$ .

**Better alternatives:**

► **Implicit (Backward Euler):**

$$u^{n+1} = u^n + \Delta t i\kappa|u^{n+1}|^2u^{n+1}$$

- More stable than explicit Euler
- Requires nonlinear solve at each grid point

► **Crank–Nicolson:**

$$u^{n+1} = u^n + \frac{\Delta t}{2} i\kappa (|u^n|^2u^n + |u^{n+1}|^2u^{n+1})$$

- Second-order accurate
- Better conservation properties
- Still implicit

- ▶ **Runge–Kutta (e.g. RK4):**
  - ▶ Fully explicit
  - ▶ Higher accuracy
  - ▶ More expensive (multiple evaluations)
- ▶ **Recommended:**

$$u(t + \Delta t) = e^{i\kappa|u(t)|^2\Delta t}u(t)$$

- ▶ Amplitude  $|u|$  is constant
- ▶ Pure phase rotation
- ▶ Unconditionally stable

## Takeaway:

### Best Practice

Because the nonlinear ODE is exactly solvable, using the exact phase update is both more stable and more accurate than Euler.

# Algorithm Overview

Given  $u^n(x)$ :

1. FFT  $\rightarrow \hat{u}^n$
2. Multiply by  $e^{-ik^2\Delta t/2}$
3. IFFT  $\rightarrow u^*$
4. Nonlinear step:

$$u^{**} = u^* + \Delta t i\kappa |u^*|^2 u^*$$

5. FFT  $\rightarrow \widehat{u^{**}}$
6. Multiply by  $e^{-ik^2\Delta t/2}$
7. IFFT  $\rightarrow u^{n+1}$

# FFT Conventions

Two common conventions:

**Mathematics:**

$$\hat{u}(k) = \int u(x) e^{-ikx} dx$$

**Numerical FFT:**

$$\hat{u}_k = \sum_{j=0}^{N-1} u_j e^{-2\pi i j k / N}$$

Key differences:

- ▶ Scaling factors
- ▶ Definition of  $k$

**Important:** Always match your  $k$  grid to FFT convention!

# Discrete Wavenumbers

For domain  $[-L, L]$ ,  $N$  points:

$$k = \frac{\pi}{L} \cdot \left\{ 0, 1, \dots, \frac{N}{2} - 1, -\frac{N}{2}, \dots, -1 \right\}$$

**Why this matters:**

- ▶ Determines  $k^2$  in phase factor
- ▶ Incorrect ordering  $\Rightarrow$  wrong evolution

# Julia FFT Example

```
using FFTW
```

```
N = 256
```

```
L = 10.0
```

```
dx = 2L/N
```

```
x = range(-L, L, length=N)
```

```
k = (pi/L) * vcat(0:N÷2-1, -N÷2:-1)
```

```
u = exp.(-x.^2)
```

```
uhat = fft(u)
```

```
u_back = ifft(uhat)
```

# Mathematica FFT Example

```
n = 256;  
L = 10;  
  
x = Subdivide[-L, L, n-1];  
  
u = Exp[-x^2];  
  
uhat = Fourier[u];  
uBack = InverseFourier[uhat];
```

**Note:** Mathematica uses a symmetric normalization convention.

# Why the Linear Step is Easy

- ▶ PDE becomes ODE system in Fourier space
- ▶ Completely decoupled modes
- ▶ Exact solution available

$$\hat{u}(k, t) = e^{-ik^2 t} \hat{u}(k, 0)$$

**Takeaway:**

Core Idea

The FFT converts a differential operator into multiplication.

# Pseudocode

Given  $u(x)$

Precompute  $k$  and  $\text{exp\_factor} = \exp(-i*k^2*dt/2)$

for each timestep:

$\text{uhat} = \text{FFT}(u)$

$\text{uhat} *= \text{exp\_factor}$

$u = \text{IFFT}(\text{uhat})$

$u = u + dt * i*kappa*abs(u)^2 * u$

$\text{uhat} = \text{FFT}(u)$

$\text{uhat} *= \text{exp\_factor}$

$u = \text{IFFT}(\text{uhat})$

end

# Summary

- ▶ Split PDE into linear + nonlinear parts
- ▶ Linear part solved exactly in Fourier space
- ▶ Nonlinear part handled explicitly
- ▶ Strang splitting gives second-order accuracy
- ▶ FFT is the key computational tool